

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Strojové učení pro adaptaci autonomních agentů

Machine learning for the adaptation of autonomous agents

VÝZKUMNÝ ÚKOL

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2024

České vysoké učení technické v Praze

Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství

Akademický rok 2023/2024

ZADÁNÍ VÝZKUMNÉHO ÚKOLU

Student: Bc. Martin Jochec

Studijní program: Aplikace informatiky v přírodních vědách

Název práce: Strojové učení pro adaptaci autonomních agentů

Pokyny pro vypracování:

1. Nastudujte a popište problematiku umělé inteligence ve 3D akčních hrách.
2. Prozkoumejte možnosti strojového učení pro průběžnou adaptaci na chování protihráče.
3. Navrhněte algoritmy pro průběžnou adaptaci agenta na chování protihráče.
4. Navrhněte metodiky hodnocení úspěšnosti agenta.

Doporučená literatura:

- [1] Liébana, D. P., Lucas, S. M., Gaina, R. D., Togelius, J., Khalifa, A., & Liu, J. (2020). General Video Game Artificial Intelligence. In Synthesis Lectures on Games and Computational Intelligence. Springer International Publishing. <https://doi.org/10.1007/978-3-031-02122-0>
- [2] SUTTON, Richard S. a Andrew G. BARTO. Reinforcement Learning: An Introduction. Bradford Book, 1998. ISBN 0262039249.
- [3] ABBEEL, Pieter a John SCHULMAN. Continuous control with deep reinforcement learning. arXiv preprint, 2016. Dostupné z: <https://arxiv.org/abs/1509.02971>.
- [4] RUSSELL, Stuart a Peter NORVIG. Artificial Intelligence: A Modern Approach. Prentice Hall, 2020. ISBN 978-0136042594.

Jméno a pracoviště vedoucího práce:

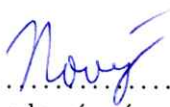
Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, ČVUT v Praze

Datum zadání výzkumného úkolu: 25. 10. 2023

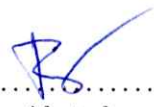
Termín odevzdání výzkumného úkolu: 25. 8. 2024

V Praze dne 25. 10. 2023

.....

vedoucí práce

.....

garant programu

.....

vedoucí katedry

Prohlášení

Prohlašuji, že jsem výzkumný úkol vypracoval samostatně a použil jsem pouze podklady (literaturu, projekty, SW atd.) uvedené v příloženém seznamu.

V Praze dne

.....
Bc. Martin Johec

Poděkování

Děkuji Ing. Josefovi Novému, Ph.D. za vedení mého výzkumného úkolu a pomoc při jeho tvorbě a také ??? za kontrolu správnosti textu výzkumného úkolu.

Bc. Martin Johec

Název práce:

Strojové učení pro adaptaci autonomních agentů

Autor: Bc. Martin Jocheč

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Výzkumný úkol

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt: Tento výzkumný úkol se zaměřuje na teorii a návrh kombinace umělé inteligence (AI) a strojového učení ve hrách za účelem dynamické úpravy chování herních protivníků v reálném čase. Úkol se skládá ze dvou částí. První část pojednává o historii a typech umělé inteligence, konkrétně o umělé inteligenci v 3D akčních hrách, a zahrnuje základy strojového učení a jeho různých algoritmů. Druhá část navrhuje algoritmy pro agenty AI v 3D akčních hrách, které citlivě přizpůsobují jejich strategie. Rovněž nastiňuje metodiky vyhodnocování výkonnosti agentů, aby bylo zajištěno přesné vyhodnocování v dynamických herních prostředích.

Klíčová slova: autonomní agent, hry, umělá inteligence, adaptace, strojové učení

Title:

Machine learning for the adaptation of autonomous agents

Author: Bc. Martin Jocheč

Abstract: This research task focuses on the theory and design of combining artificial intelligence (AI) and machine learning in games to dynamically adjust the behavior of game opponents in real time. The task consists of two parts. The first part discusses the history and types of AI, specifically AI in 3D action games and covers the basics of machine learning and its various algorithms. The second part proposes algorithms for AI agents in 3D action games to adapt their strategies sensitively. It also outlines methodologies for evaluating agent performance to ensure accurate evaluation in dynamic game environments.

Key words: autonomous agent, games, artificial intelligence, adaptation, machine learning

Obsah

Úvod	9
1 AI ve hrách	11
1.1 Historie	11
1.2 „Chování pro tento případ“ neboli Ad-Hoc Behavior	14
1.2.1 Konečný automat (Finite State Machines)	14
1.2.2 Stromy chování (Behavior Trees)	16
1.2.3 „AI založená na užitečnosti“ neboli Utility System AI	17
1.3 Příklady dalších metod	19
1.3.1 Procházení stromu	19
1.3.2 Evoluční algoritmy	19
1.4 AI ve 3D akčních hrách	20
1.4.1 Obecné úkoly AI	20
1.4.2 F.E.A.R.	20
1.4.3 ME: Shadow of Mordor a ME: Shadow of War	22
2 Strojové učení (Machine Learning - ML)	25
2.1 Supervised Machine Learning	26
2.1.1 Klasifikace	27
2.1.2 Regrese	30
2.2 „Učení bez učitele“ neboli Unsupervised ML	32
2.3 „Částečně řízené učení“ neboli Semi-Supervised ML	33
2.4 „Zpětnovazebné učení“ neboli Reinforcement Learning	34
3 Návrh autonomního agenta	35
3.1 Prostředí pro agenta	35
3.2 Různé algoritmy pro učení agenta	36
3.2.1 Pomocí SARSA (State-Action-Reward-State-Action)	37
3.2.2 Pomocí Deep Q-Learning	38
4 Různé návrhy hodnocení agenta	39
4.1 Podle efektivity zabití nepřátel	39
4.2 Podle efektivity přežití	40
4.3 Efektivita zabití + přežití	41
Závěr	43
Literatura	44

Úvod

Tento výzkumný úkol se zabývá teorií a návrhem propojení umělé inteligence (AI) a strojového učení ve hrách k dynamickému přizpůsobování chování protivníků v reálném čase.

V první kapitole jsou obsaženy dvě části. První část popisuje umělou inteligenci ve hrách. Specificky popisuje historii umělé inteligence ve stolních a digitálních hrách, dále rozebírá některé algoritmy, které jsou nejčastěji používány k vytvoření umělé inteligence v digitálních hrách, ale díky kterým není umělá inteligence schopna se přizpůsobit nebo upravit svoje chování. Nakonec se tato část specificky zaměří na některé existující 3D akční hry, které obsahují inteligentnější AI. Druhá část obsahuje základy strojového učení, existující typy strojového učení a také některé algoritmy z těchto typů.

Kapitola 1

AI ve hrách

1.1 Historie

Prolínání her a umělé inteligence (AI) je dlouhodobě zkoumanou oblastí. Velká pozornost ve výzkumu AI týkající se her se zaměřuje na vývoj agentů schopných zapojit se do hry, ať už s integrací strojového učení, nebo bez ní. Tato cesta zkoumání historicky představovala primární a často jediný přístup k využití umělé inteligence v herním kontextu.

Ještě před formalizací umělé inteligence jako samostatného oboru se průkopníci počítačové vědy snažili prozkoumat možnosti výpočetní techniky tím, že navrhovali programy schopné hrát stolní hry. Mezi těmito prvními snahami je práce Alana Turinga, který je obecně považován za zásadní postavu počítačové vědy a který upravil algoritmus Minimax pro hraní šachů.

Počátky zapojení umělé inteligence do digitálních her lze vysledovat až k průkopnické práci A.S. Douglase z roku 1952, kdy v rámci své doktorské práce na univerzitě v Cambridge vyvinul software schopný naučit se dokonale hrát Tic-Tac-Toe. Příspěvky Arthura Samuela následně znamenaly významný milník v podobě zavedení takzvaného „reinforcement“ učení, předchůdce současných technik strojového učení, demonstrovaného na programu, který autonomně zlepšoval svojí taktiku v dámě prostřednictvím opakovaného samohraní[1].

Raný výzkum v oblasti umělé inteligence zaměřené na hraní her se převážně soustředil na tradiční deskové hry, jako je dáma nebo šachy. Tyto hry byly obzvláště zajímavé díky své schopnosti generovat složité komplexy z jednoduchých souborů pravidel a díky svému historickému postavení po staletí testovala hranice lidského poznání. V roce 1994 došlo k významnému průlomů, když program Chinook Checkers porazil úřadujícího mistra světa v dámě Mariona Tinsleyho. Následně v roce 2007 byla dáma zcela vyřešena[2], což znamenalo významný milník ve výzkumu AI.

Šachy, často označované jako „octomilka umělé inteligence“, posloužily jako klíčové měřítko pro testování nových metod umělé inteligence. Vývoj softwaru, který je schopen překonat lidský výkon v šachu, však posunul zaměření výzkumu AI na jiné oblasti. Deep Blue[4] od společnosti IBM, využívající algoritmus Minimax rozšířený o úpravy specifické pro šachy a vysoce optimalizovanou funkci vyhodnocování šachovnic, v roce 1997 slavně porazil mistra světa Garryho Kasparova.

Dalším významným pokrokem, který předcházel úspěchům Deep Blue a Chinook, je demonstrován softwarem TD-Gammon[3], který v roce 1992 vyvinul Gerald Tesauro. TD-Gammon se vyznačuje využitím umělé neuronové sítě vycvičené pomocí „temporal difference learning“, která se sama proti sobě opakovaně zapojuje do hry. Pozoruhodné je, že TD-Gammon dosáhl dovedností, které jsou srovnatelné s uznávanými lidskými hráči vrhcábu, což dokazuje účinnost jeho tréninkové metodiky.

Po průlomových objevech v oblasti umělé inteligence v tradičních deskových hrách dosáhla vrcholu v oblasti umělé inteligence v deskových hrách hra Go v roce 2016. Hra Go se stala novou hračkou pro herní umělou inteligenci, která se vyznačuje bezkonkurenčním faktorem větvení blížícím se k číslu 250 a rozsáhlým prohledávacím prostorem, který řádově převyšuje prostor šachů. Ze začátku bylo dosažení lidské úrovně v Go považováno za vzdálenou budoucnost, ale nakonec v říjnu 2015 odehrál AlphaGo, který patří společnosti Google DeepMind a je vybaven revolučním frameworkem „Deep reinforcement learning“, svůj první zápas proti úřadujícímu trojnásobnému mistru Evropy Fan Hui, který i vyhrál a skóroval 5:0. V roce 2016 Lee Sedol, profesionální hráč Go s devíti tituly, podlehl porážce v zápase na pět her proti AlphaGo. Díky těmto drtivým vítězstvím se hra Go stala posledních stolní hrou, kde počítače dosáhly nadlidských schopností v klasických deskových hrách, protože další stolní hry zatím lidské hráče v širokém měřítku nezaujaly.[5]

Tradiční deskové hry, které se vyznačují strukturovaným tahovým hraním a transparentním sdílením informací mezi hráči, však představují pouze podmnožinu herních prostředí. Uznává se, že inteligence zahrnuje širší dimenze než ty, které jsou testovány v těchto tradičních hrách.

V posledních patnácti letech se vytvořila rostoucí komunita, která se zaměřuje především na zlepšování schopností umělé inteligence v digitálních hrách s cílem dosáhnout výkonnosti srovnatelné s lidskými hráči, replikovat specifické lidské strategie nebo vykazovat jedinečné vlastnosti.

K významnému pokroku v této oblasti došlo v roce 2014, kdy algoritmy vyvinuté společností Google DeepMind prokázaly schopnost vyniknout v různých hrách z klasické konzole Atari 2600 a překonaly výkon na úrovni člověka pouze za použití surových pixelových vstupů. Mezi těmito hrami představovala výzvu pro konvenční přístupy hra Ms. Pac-Man (Namco, 1982). Tým Microsoft Maluuba však tuto výzvu vyřešil použitím techniky posilovacího učení s hybridní architekturou odměn, čímž nakonec dosáhl řešení této hry.

V poslední době však došlo k prudkému nárůstu výzkumu zaměřeného na vývoj věrohodných agentů ve hrách, což otevřelo cestu novým směrům v herní umělé inteligenci.

Jednou z cest zkoumání je vytváření agentů schopných projít herním testem, který se jmenuje Turing test[6]. Tento test, odvozený od klasického Turingova testu, vyžaduje, aby porota přesně rozeznala, zda pozorované herní chování pochází od lidského hráče nebo od herního bota řízeného umělou inteligencí. V roce 2012 se dva boty řízené umělou inteligencí ve hře Unreal Tournament 2004 od Epic Games úspěšně prokázaly schopnost projít herním Turingovým testem, což svědčí o jejich schopnosti replikovat herní chování podobné lidskému.

1.2 „Chování pro tento případ“ neboli Ad-Hoc Behavior

V oblasti vývoje her je již dlouho známé tzv. „Ad-Hoc Behavior“, jako jsou konečné stavové stroje, stromy chování a umělá inteligence založená na užitku. Tyto metody tradičně umožňují značnou kontrolu nad nehráčskými postavami (NPC). O jejich trvalé převaze svědčí i to, že se s jejich používáním v komunitě vývojářů her stále spojuje termín „herní umělá inteligence“.

1.2.1 Konečný automat (Finite State Machines)

Až do poloviny 20. století byl dominantní metodou pro ovládání a rozhodování nehráčských postav (NPC) ve hrách konečný automat (Finite State Machines, KA) [7] a jeho varianty, například hierarchické KA. Tyto techniky měly v oblasti herní umělé inteligence převahu a prokázaly svou účinnost při řízení chování NPC v herním prostředí.

Konečné automaty spadají do oblasti expertních znalostních systémů a běžně se vizualizují jako grafy. Tyto grafy slouží jako abstraktní reprezentace vzájemně propojených prvků, jako jsou objekty, symboly, události, akce nebo vlastnosti relevantní pro konkrétní kontext návrhu. V rámci této reprezentace uzly odpovídají stavům, z nichž každý je charakterizován matematickou abstrakcí, zatímco hrany označují přechody, které znamenají podmíněné vztahy mezi stavy. KA fungují tak, že v daném okamžiku může být aktivní pouze jeden stav, přičemž k přechodům dochází na základě splnění předem definovaných podmínek. V podstatě jsou charakterizovány třemi základními prvky:

1. Stav:

- Obsahují informace týkající se úlohy, jako je například aktuální stav „zkoumání“.

2. Přechody:

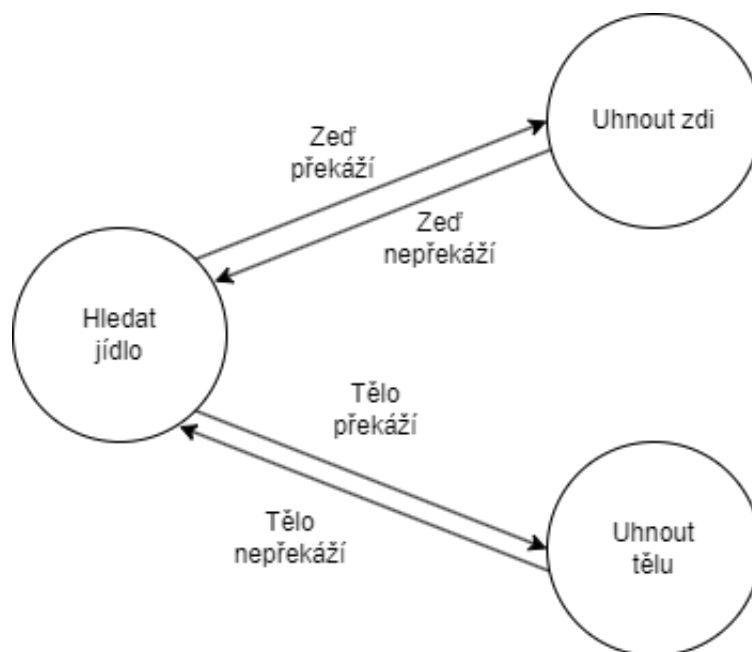
- Označují změny stavu vyvolané podmínkami, které musí být splněny, jako je přechod do stavu „upozorněn“ po zaslechnutí střelby.

3. Akce:

- Určují specifické chování, jež má být provedeno, například náhodný pohyb a hledání protivníka ve stavu „průzkum“.

Konečné automaty nabízejí pozoruhodné výhody díky své jednoduchosti při návrhu, implementaci, vizualizaci a ladění. Při aplikaci na rozsáhlejší systémy však mohou představovat výzvu kvůli své potenciální složitosti, což vede k výpočetním omezením v některých aspektech.

Kromě toho mají významnou nevýhodou, spolu s dalšími ad-hoc behavior metodami, a to je jejich přirozená nedostatečná flexibilita a dynamika, pokud nejsou speciálně navrženy tak, aby tyto vlastnosti zahrnovaly. Jakmile jsou dokončeny, otestovány a odladěny, jejich přizpůsobivost a evoluční kapacita jsou omezeny, což vede k projektům relativně předvídatelného chování v herním kontextu.



Obrázek 1.1: Příklad KA pro jednoduchou hru Snake. Obsahuje tři stavy a čtyři přechody. Had se postupně přibližuje k jídlu, ale pokud má před sebou zeď a nebo svoje tělo, tak se těmito prvky pokusí vyhnout.

1.2.2 Stromy chování (Behavior Trees)

Strom chování (Behavior Tree, SC)[8] funguje jako expertní znalostní systém, podobně jako konečný automat (Finite State Machine, KA), který usnadňuje přechody mezi konečnou množinou úloh nebo chování. Významná výhoda SC oproti KA spočívá v jejich modulární struktuře, která umožňuje skládání složitých chování z jednodušších úloh. Na rozdíl od KA, které představují především stavy, jsou SC strukturovány kolem chování, čímž se odlišují od hierarchických KA. Podobně jako KA nabízejí SC snadný návrh, testování a ladění, což přispělo k jejich širokému rozšíření při vývoji her po úspěšných implementacích v titulech, jako je Halo 2[9] a Bioshock.

Využívají hierarchickou stromovou strukturu, která se skládá z kořenového uzlu, nadřazených uzlů a jim odpovídajících podřízených uzlů, z nichž každý představuje specifické chování (viz obrázek 1.2). Procházení SC začíná od kořene a aktivuje provádění dvojic rodič-dítě, jak je ve stromu naznačeno. Během předem stanovených časových kroků může podřízený uzel vrátit svému rodiči jednu z následujících hodnot: „běží“, pokud chování zůstává aktivní, „úspěch“ při úspěšném dokončení nebo „selhání“, pokud chování selže. SC se skládají ze tří základních typů uzlů: sekvence, selektor a dekorátor, z nichž každý plní odlišné funkce, které jsou podrobněji popsány níže:

1. Sekvence (na obr.1.2 znázorněna šedou barvou):

- Úspěch v podřízeném chování vyvolá pokračování v sekvenci, což nakonec vede k úspěchu nadřazeného uzlu, pokud všechna podřízená chování uspějí; naopak, sekvence selže, pokud některé podřízené chování selže.

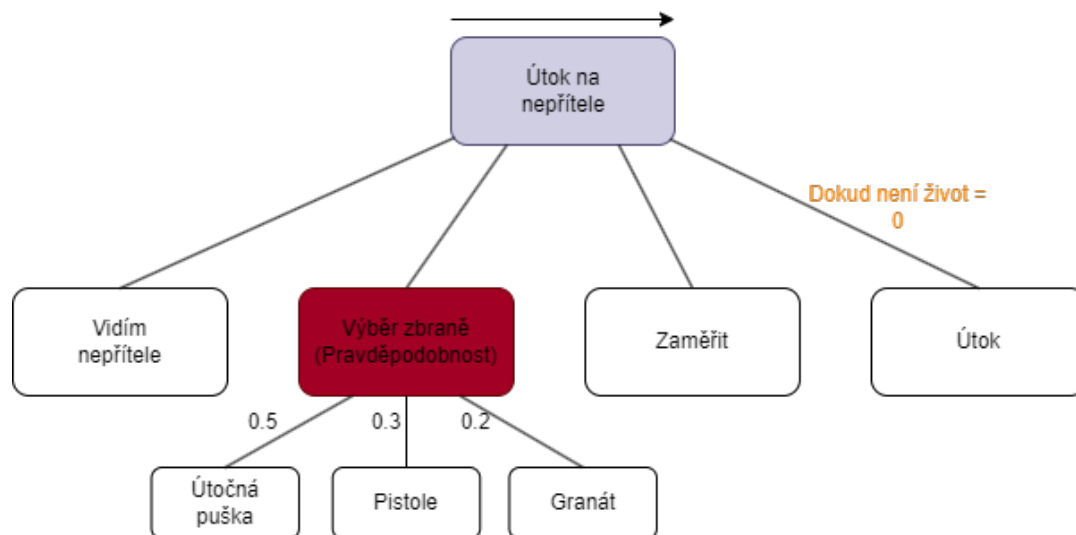
2. Selektor (na obr.1.2 znázorněn červenou barvou):

- Uzly selektorů zahrnují dva základní typy: selektory pravděpodobnosti a selektory priority. U pravděpodobnostních selektorů jsou podřízená chování vybírána na základě pravděpodobností předem stanovených tvůrcem SC. Naopak prioritní selektory řadí podřízená chování do seznamu a zkoušejí je postupně. Bez ohledu na typ selektoru vede úspěch v podřízeném chování k úspěchu selektoru. Při neúspěchu v podřízeném chování je vybráno další podřízené chování v pořadí (u prioritních selektorů) nebo selže samotný selektor (u pravděpodobnostních selektorů).

3. Dekorátor (na obr.1.2 znázorněn oranžovou barvou):

- Uzly dekorátoru rozšiřují složitost a možnosti jednotlivých podřízených chování. Mezi příklady dekorátorů patří určení počtu iterací, které podřízené chování podstoupí, nebo přidělení časového limitu pro jeho dokončení.

Ve srovnání s KA nabízejí větší flexibilitu při navrhování a jsou relativně jednodušší na testování. Nicméně mají podobná omezení, zejména pokud jde o jejich statickou povahu, která omezuje jejich přizpůsobivost a dynamičnost. Zatímco pravděpodobnostní výběrové uzly vnášejí prvek nepředvídatelnosti, jsou snahy o zvýšení adaptability prostřednictvím úprav ve struktuře stromů.



Obrázek 1.2: Příklad stromu chování

1.2.3 „AI založená na užitečnosti“ neboli Utility System AI

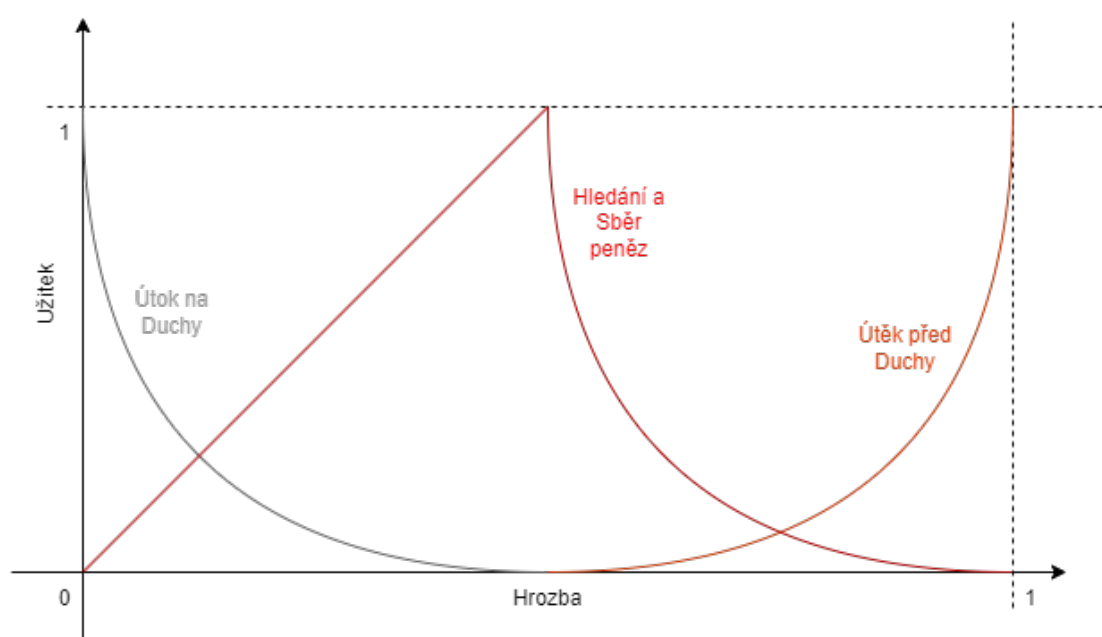
V odvětví herní umělé inteligence je velká absence modularity chování v různých hrách a představuje významnou překážku pro dosažení vysoce kvalitní umělé inteligence. K překonání omezení KA a SC v tomto ohledu je stále oblíbenějším přístupem umělá inteligence založená na užitečnosti (Utility System AI, USAI)[10]. Tato metoda nabízí prostředky pro efektivnější návrh řídicích a rozhodovacích systémů ve hrách. V USAI je ke každé herní instanci přiřazena funkce užitku, která kvantifikuje její důležitost v kontextu hry. Tyto funkce mohou například hodnotit faktory, jako je blízkost nepřítele nebo zdravotní stav agenta. Na základě zvážení všech dostupných užitečných hodnot a potenciálních akcí určí umělá inteligence nejzásadnější možnost, kterou je třeba v daném okamžiku upřednostnit. Tento přístup vychází z principů teorie užitku z ekonomie a zahrnuje návrh funkcí užitku, podobně jako vytváření funkcí příslušnosti v teorii fuzzy množin.

Užitky zahrnují široké spektrum ukazatelů, od objektivních údajů, jako je zdraví nepřítele, až po subjektivní faktory, jako jsou emoce a vnímané hrozby. Tyto různorodé metriky spojené s potenciálními akcemi nebo rozhodnutími lze sloučit do lineárních nebo nelineárních vzorců, které slouží k vedení agentů při rozhodování na základě souhrnného skóre užitečnosti. Na rozdíl od sekvenčních rozhodovacích procesů KA a SC umožňuje USAI současné vyhodnocení všech dostupných možností. Přiřazením užitku každé možnosti a výběrem té s nejvyšším užitekem mohou agenti určit nejvhodnější postup.

Tyto hodnoty užitku podléhají pravidelnému přehodnocování, například přepočty probíhající v předem stanovených intervalech během hry, aby se zajistilo sladění s dynamickým stavem hry.

USAI má oproti jiným technikám ad-hoc chování výrazné výhody. Její modularita umožňuje herním agentům rozhodovat se na základě dynamického souboru faktorů, což podporuje adaptabilitu rozhodovacích procesů. Rozšiřitelnost navíc umožňuje bezproblémovou integraci nových úvah podle potřeby.

Opakovaná použitelnost metody navíc usnadňuje přenos užitekových komponent mezi rozhodnutími a napříč různými hrami, což zvyšuje efektivitu vývoje. Tento přístup se hojně využívá v různých herních žánrech, přičemž jeho významné implementace byly zaznamenány ve hrách, jako je Red Dead Redemption pro výběr zbraní a dialogů, a také ve hře F.E.A.R. pro dynamické taktické rozhodování[11].



Obrázek 1.3: Řízení chování hry Pac-Man založené na USAI. Hrozba (osa x) je funkce, která leží mezi 0 a 1 a je založena na aktuální pozici duchů. Pac-Man bere v úvahu aktuální úroveň ohrožení, přiřazuje hodnoty užitku (prostřednictvím tří různých křivek) a rozhodne se sledovat chování s nejvyšší hodnotou užitku. V tomto příkladu roste exponenciálně útěk před duchy s ohledem na ohrožení. Naopak útok na duchy exponenciálně klesá s ohledem na ohrožení. Nakonec hledání a sběr peněz roste lineárně až do určité úrovně ohrožení, od tohoto bodu klesá exponenciálně.

1.3 Příklady dalších metod

1.3.1 Procházení stromu

Procházení stromem[12] je základní úlohou v teorii algoritmizace a zahrnuje systematické zkoumání všech uzlů v rámci datové stromové struktury. Vzhledem k analogii stromu s grafem je procházení stromů úzce spjato s teorií grafů. Metody procházení stromů se běžně demonstrují na binárních stromech, ale lze je snadno rozšířit i na jiné stromové struktury. Dva základní přístupy, prohlédávání do hloubky (DFS) a prohlédávání do šířky (BFS), se liší pořadím návštěv uzlů a použitím pomocných datových struktur. DFS, často rekurzivní, obvykle využívá zásobník, a to buď explicitně, nebo implicitně prostřednictvím použití zásobníku volání. Naproti tomu BFS typicky využívá frontu.

Ve scénářích, kde je cílem cílené vyhledávání hodnot spíše než úplné procházení stromu, umožňují specializované konstrukční techniky, jako je vytváření vyhledávacího stromu (typicky binárního vyhledávacího stromu), efektivní použití přizpůsobených variant algoritmů. Další specializované algoritmy jsou použitelné v problémech prohlédávání stavového prostoru, které lze reprezentovat jako stromy. Pozoruhodné je, že heuristické algoritmy, včetně stromového vyhledávání Monte Carlo odvozeného z metody Monte Carlo, našly úspěch v aplikacích umělé inteligence v rámci počítačových her.

1.3.2 Evoluční algoritmy

Evoluční algoritmy (EA)[13] jsou podmnožinou evolučních výpočtů a představují metaheuristické optimalizační algoritmy založené na populaci. Tyto algoritmy čerpají inspiraci z biologické evoluce a využívají mechanismy, jako je reprodukce, mutace, rekombinace a selekce. V EA představují kandidáti na řešení optimalizačních problémů jedince v populaci, jejichž kvalita se hodnotí pomocí fitness funkce. Iterativním použitím těchto operátorů se populace vyvíjí. Výpočetní složitost, zejména spojená s vyhodnocováním fitness funkcí, často představuje v reálných implementacích EA značnou výzvu. K řešení tohoto problému se používají metody, jako je aproximace fitness. Navzdory své zdánlivé jednoduchosti vykazují EA účinnost při řešení složitých problémů, což naznačuje, že složitost algoritmu nemusí vždy přímo korelovat se složitostí problému.

1.4 AI ve 3D akčních hrách

1.4.1 Obecné úkoly AI

V 3D akčních hrách by se měla umělá inteligence umět pohybovat virtuálním světem, takže například ovládat základní pohyby po mapě ale také pokročilé strategie jako například vylézt po zdi nebo skrčit se a projít ventilační šachtou a tak dále. Klíčové jsou také poznávací schopnosti a přizpůsobivost, neboli například dokázat předvídat hráčovi interakce s objekty nebo se samotnou umělou inteligencí. AI by mělo také vynikat v boji a používat různé taktiky, jako je obcházení hráče a nebo koordinované útoky. Kromě boje by měli zvládat i úkoly, jako je interakce s prostředím nebo pomáhat hráčovi dosáhnout cíle. Pokud hra obsahuje i příběhovou stránku, tak by měla AI umět mimikovat emoce, aby dokázala vyjadřovat řadu pocitů, aby hráč se vcítil do své postavy a tím pádem do herního světa.

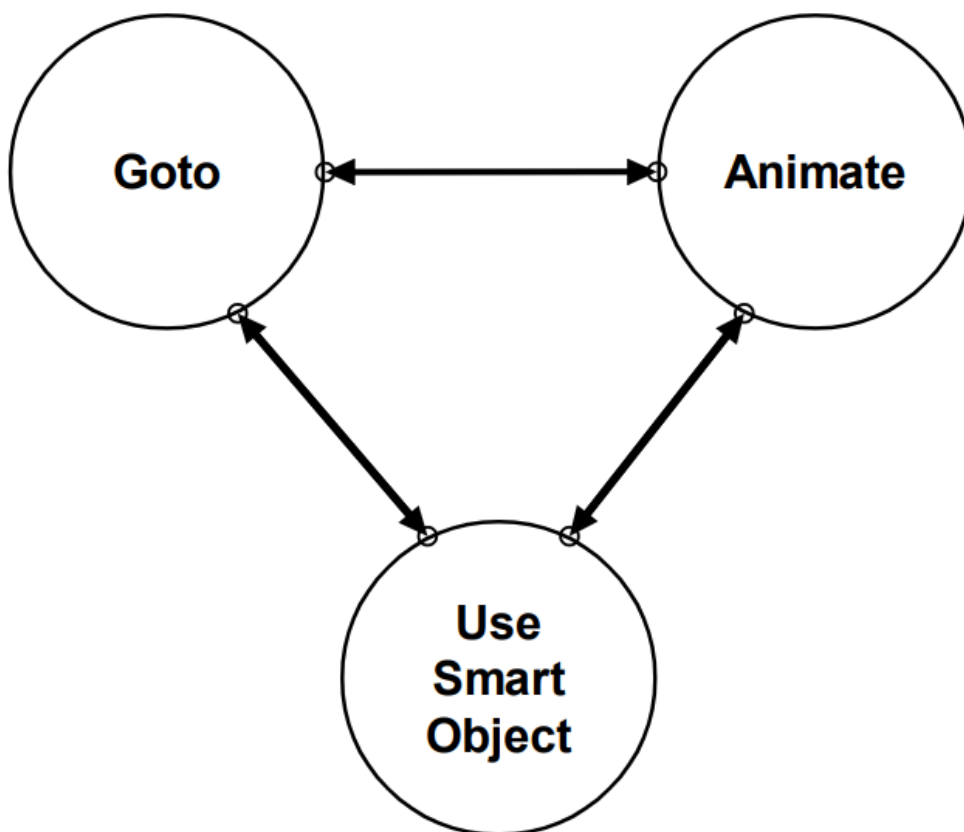
1.4.2 F.E.A.R.

Hra F.E.A.R.[14][15], zkratka pro First Encounter Assault Recon, debutovala v roce 2005 jako střílečka z pohledu první osoby vyvinutá společností Monolith Productions. Navzdory svému stáří je stále chválena jako ukázkový příklad implementace umělé inteligence v bojových videohrách. Klíč k jejímu trvalému úspěchu spočívá v kombinaci inovativních technik umělé inteligence a strategického herního designu.

Jako první do hry implementovali techniku umělé inteligence zvanou automatizované plánování. Tento přístup, inspirovaný systémem STRIPS[16] vyvinutým v roce 1971 pro programování robotů, umožňuje nehráčským postavám (NPC) zapojit se do sofistikovaných rozhodovacích procesů v herním prostředí. Konkrétně jsou NPC vybaveny schopností plánovat svůj pohyb, vybírat krytí a rozhodovat o bojových akcích, jako je střelba nebo použití granátu. Tato integrace automatizovaného plánování zahrnuje přiřazení konkrétních cílů každé NPC a modelování možných akcí, které mají v herním světě k dispozici. Analýzou těchto možností mohou NPC vytvářet optimalizované plány pro efektivní dosažení svých cílů.

Každá akce umělé inteligence ve F.E.A.R. se řídí předpoklady a účinky. Předpoklady představují podmínky, které jsou ve světě hry nezbytné pro provedení akce, zatímco účinky označují výsledky vyplývající z provedení akce. Například, pokud je postava pozorována, jak běží k objektu a sráží jej na zem, aby jej použila jako úkryt, znamená to sekvenci plánovaných akcí: pohyb, manipulace s objektem a nakonec hledání úkrytu. To znamená, že před zahájením sekvence akcí je třeba zajistit, aby byl objekt vzprámený, dosažitelný a schopný poskytnout úkryt. Dodržování předpokladů zabráňuje nelogickým akcím, jako je pokus o manipulaci s objektem, který je již převrácený, nebo pohyb k nedosažitelnému objektu, čímž se zvyšuje realističnost NPC ve hře.

Kromě plánovacích funkcí využívá systém F.E.A.R. ke koordinaci provádění formulovaných strategií konečný automat (KA). Programátoři v Monolithu si uvědomily, že mohou mapovat každou akci umělé inteligence ve hře na odpovídající animaci, ať už jde o střelbu ze zbraně, přemísťování nebo interakci s prvky prostředí. Díky tomuto se zjednodušuje složitost chování AI tím, že v rámci KA rozděluje akce do tří různých stavů: pohyb na určené pozice, animační sekvence a specializované interakce s určenými „inteligentními objekty“. Bez ohledu na složitost taktického plánu umělé inteligence jej lze vždy zahrnout do těchto tří stavů KA, což zajišťuje zjednodušené provádění a udržuje soudržnost akcí NPC v průběhu hry.



Obrázek 1.4: Konečný automat hry F.E.A.R.[17]

Dalším aspektem designu AI, kterou vývojáři implementovali, je kdy využít AI k dosažení požadovaných výsledků a kdy použít důmyslná řešení a úpravy, neboli „padělat“ chytrost AI. Například zatímco jednotliví vojáci ve hře F.E.A.R. nemají povědomí o existenci ostatních, hra vytváří iluzi týmové spolupráce pomocí chytré manipulace. Přidělením samostatných cílů různým postavám, jejichž kombinace simuluje spolupráci, hra naaranžuje scénáře, v nichž například jedna postava poskytuje krycí palbu, zatímco druhá se postaví do boku hráče, aby ho mohli jednodušeji zneškodnit. Ačkoli jsou tyto akce prováděny nezávisle na sobě, jejich synchronizace vytváří dojem koordinovaného manévru. Pro posílení této iluze hra obsahuje dialogy mezi postavami, přestože se navzájem neslyší a neodpovídají si, což přidává další vrstvu realismu a umocňuje hráčův dojem, že bojuje proti koordinovanému týmu.

1.4.3 ME: Shadow of Mordor a ME: Shadow of War

Systém Nemesis[19], představený ve hře Middle-Earth: Shadow of Mordor v roce 2014, který stále patří mezi nejvyspělejší AI ve videohrách. V podstatě funguje jako dynamický generátor padouchů, který vytváří unikátní protivníky s odlišnými vlastnostmi, osobnostmi a rysy. Tito protivníci hráče zaujmou posměšnými průpovídkami a přispívají k příběhu, který se vyvíjí na základě interakcí s hráčem.

V Middle-Earth: Shadow of War je systém Nemesis druhé generace, který rozšiřuje systém z předchozí hry v mnoha ohledech. Nejenže generuje padouchy, ale také spřádá složité příběhy založené na vztazích hráče s těmito orky. Tento systém důmyslně využívá základní hráčské akce, jako je boj, útěk nebo shromažďování informací, jako stavební kameny příběhu a vytváří tak dynamický příběhový zážitek v rámci akční hry v reálném čase.

Klíčem k úspěchu v systému Nemesis je vytvoření zapamatovatelných postav se silnou osobností a jedinečnými vlastnostmi. Díky dialogům, jako jsou například urážky mířené na hráčskou postavu nebo zpívání, ale i díky vizuálním podnětům se každý ork stává osobitým a zapamatovatelným.



Obrázek 1.5: Příklad pevnosti ve hře Shadow of War. Každý ork je unikátní jak vzhledově i chováním. Orkové s modrou lebkou jsou na straně hráče, všichni ostatní jsou nepřátelé. Tento obrázek ukazuje i hierarchii orků, kde ork na nejvyšší pozici je „Overlord“, prostřední pozice je „Warchief“ a orkové před hradbami jsou kapitáni, ale také jsou ve hře orkové, co nemají žádné postavení a jsou to buďto vojáci nebo otroci [18]

Genialita tohoto systému také spočívá v jeho schopnosti prodloužit tyto příběhy za hranice jednotlivých střetnutí a nabídnout alternativní výsledky, jako je ústup, zrada nebo dokonce vzkříšení poražených orků. To zajišťuje, že příběhy přetrvávají a vyvíjejí se na základě akcí a rozhodnutí hráče. Mezi dalšími rozšířeními těchto příběhů jsou vzácné události, jako je zrada „Warchief“ nebo proměna.

Pro podporu těchto vyvíjejících se příběhů hra také obsahuje hierarchii orkské společnosti, která umožňuje povyšování, degradování a posuny moci mezi orkami. Navíc se tento systém dynamicky přizpůsobuje na základě výkonu hráče a přináší výzvy nebo příležitosti k udržení příběhového napětí. Systém Nemesis prosperuje na základě zapojení a interpretace hráče, což umožňuje vznik různých příběhů, díky kterým vznikají individuálních zkušenosti a herní prožitky.

Kapitola 2

Strojové učení (Machine Learning - ML)

Strojové učení, jedna z hlavních součástí umělé inteligence, se zabývá vývojem modelů a algoritmů, které umožňují počítačům autonomně se učit z dat a iterativně zvyšovat svůj výkon na základě předchozích zkušeností, a to bez explicitního programování pro každou úlohu.

Zjednodušeně řečeno, strojové učení zahrnuje trénink systémů, které simulují kognitivní procesy podobné lidským, a to prostřednictvím přijímání a analýzy dat. Tato metodika tréninku zahrnuje neustálé zdokonalování prostřednictvím vystavení historickým datům, což vede k opakovanému zlepšování výkonu systému v průběhu času.

Použití strojového učení je zvláště významné v oblastech, kde existuje obrovské množství dat, která vyžadují pomoc při předvídání, což pomáhá při dosahování rychlosti a přesnosti. Tato schopnost usnadňuje práci a ukazuje tak transformační potenciál strojového učení v současných technologických ekosystémech.

Existuje několik typů strojového učení, přičemž každý z nich má zvláštní vlastnosti a aplikace:

1. Supervised Machine Learning („Učení s učitelem“)
2. Unsupervised Machine Learning („Učení bez učitele“)
3. Semi-Supervised Machine Learning („Částečně řízené učení“)
4. Reinforcement Learning („Zpětnovazebné učení“)

2.1 Supervised Machine Learning

„Učení s učitelem“ neboli Supervised Machine Learning(SML)[20] představuje algoritmický proces zaměřený na aproximaci základní funkce, která spojuje označená data s jejich odpovídajícími atributy nebo rysy. Tento metodický přístup je založen na využití označených příkladů k rozpoznání složitých vztahů v rámci souborů dat.

Vezměme si typický příklad zahrnující stroj, který má za úkol rozlišovat mezi jablky a hruškami na základě specifikovaných vlastností, jako je např. barva a velikost. Prostřednictvím vystavení mnoha označeným vzorkům ovoce - z nichž každý obsahuje odlišné kombinace barvy a velikosti spolu s odpovídajícími označeními (jablko nebo hruška) - prochází stroj fázi učení, aby zvládl tento klasifikační úkol. Po dokončení procesu učení stroj usiluje o to, aby dokázal určit kategorii (jablko nebo hruška) nového, dosud neviděného ovoce výhradně na základě pozorovaných atributů barvy a velikosti.

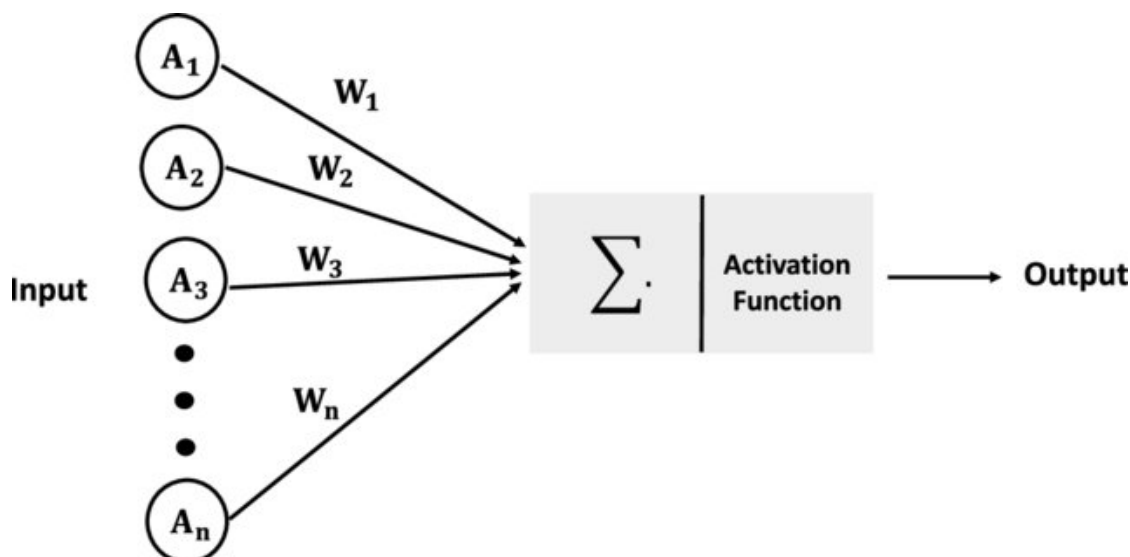
SML je zásadně závislý na přizpůsobeném souboru dat s označenými trénovacími příklady, což je vlastnost, která definuje jeho povahu pod dohledem. Konkrétně toto paradigma učení funguje tak, že dostává tréninkový signál v podobě kontrolovaných značek aplikovaných na datové instance, které jsou charakterizovány odpovídajícími rysy.

Každá datová instance v SML se skládá z dvojice složené z označených výstupů a jim přiřazených vstupních rysů. Primární cíl SML však přesahuje pouhou asimilaci dat - jeho cílem je odvodit funkci, která přesně modeluje základní vztah mezi vstupy a výstupy a která dokáže dobře zobecnit na dosud neznámé dvojice vstupů a výstupů, což je klíčový aspekt označovaný jako zobecnění.

Jako příklad praktického využití SML uveďme tyto scénáře s mapováním vstupů a výstupů:

1. {zdraví hráče, vlastní zdraví, vzdálenost k hráči} $\rightarrow$ {akce (střelba, útěk, nečinnost)}
2. {předchozí pozice hráče, aktuální pozice hráče} $\rightarrow$ {příští pozice hráče}
3. {počet zabití a headshotů, spotřebovaná munice} $\rightarrow$ {hodnocení dovednosti}
4. {skóre, prozkoumaná/neprozkoumaná část mapy} $\rightarrow$ {lokace základny hráče}

Tato mapování ilustrují složité vztahy mezi vstupy a výstupy a ukazují všestrannost a význam SML v různých oblastech, včetně her i mimo ně. Dříve než se budeme věnovat konkrétním algoritmům je nezbytné zdůraznit význam „štítkových“ dat pro utváření výstupních charakteristik a tím pádem vhodný výběr algoritmu SML.



Obrázek 2.1: Ilustrace umělého neuronu. Do neuronu se vkládá n počet vstupních vektorů $\mathbf{A}$ s odpovídajícími váhovými hodnotami $\mathbf{w}$. Neuron zpracovává vstup tak, že vypočítá vážený součet vstupů s odpovídajícími vahami a přidá váhu zkreslení (b): $x \cdot w + b$. Výsledný vzorec se vloží do aktivační funkce (g), jejíž hodnota určuje výstup neuronu.[21]

Algoritmy SML lze rozdělit do dvou hlavních typů podle povahy značkovacích dat (výstupů):

1. **Klasifikace**
2. **Regrese**

2.1.1 Klasifikace

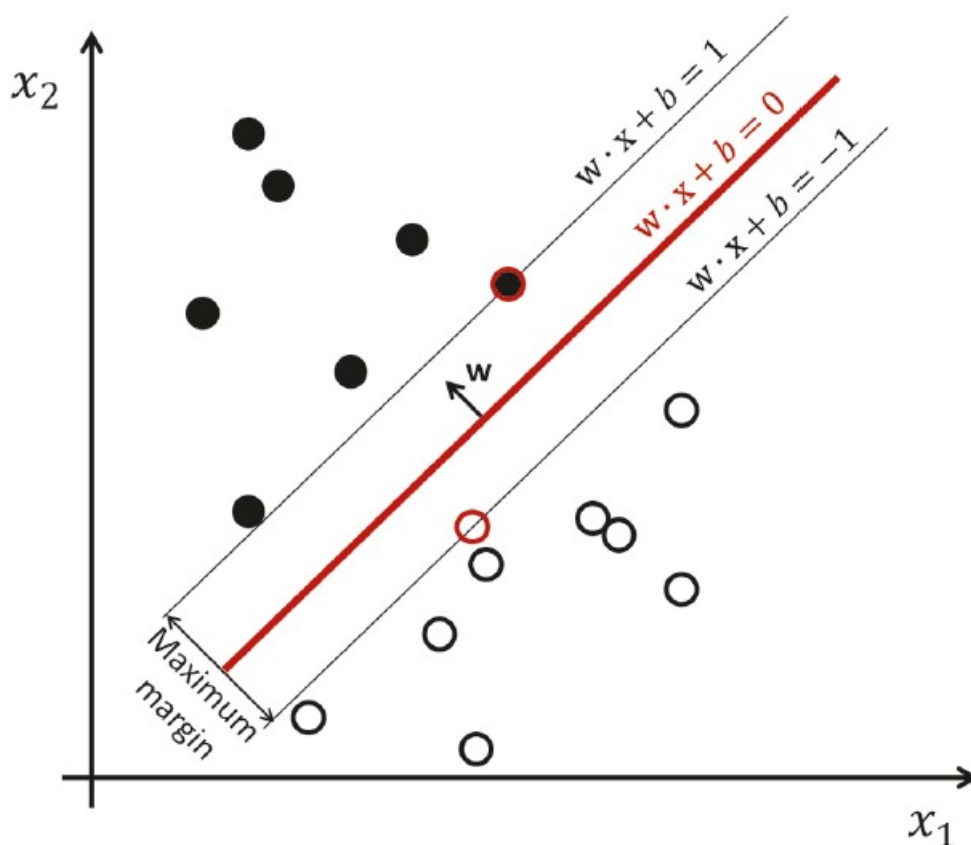
Klasifikace zahrnuje predikci kategoriálních cílových proměnných, které označují diskrétní třídy nebo štítky. Příkladem může být rozlišení e-mailů na poštu a spam, výše zmíněný příklad s jablky a hruškami a posouzení, zda pacient vykazuje vysoké riziko srdečního onemocnění. Úkolem klasifikačních algoritmů je naučit se mapování mezi vstupními znaky a předem definovanými třídami.

Support Vector Machine(Metoda podpůrných vektorů)

Support Vector Machines(SVM)[22] jsou poměrně oblíbené v oblasti algoritmů SML, které se zabývají jak klasifikačními, tak regresními úlohami. SVM fungují jako binární lineární klasifikátory, které jsou vycvičeny tak, aby optimalizovaly rozpětí mezi různými příklady tříd v rámci souboru dat, jako jsou například jablka a hrušky. Stejně jako ostatní metody SML využívají SVM atributy z nových a dosud nezaznamenaných příkladů k předpovědi příslušných tříd.

SVM jsou široce používány v různých oblastech, včetně kategorizace textu, rozpoznávání řeči, klasifikace obrázků a rozpoznávání ručně psaných znaků, a prokazují tak všestrannost a účinnost v různých aplikacích.

Podobně jako umělé neuronové sítě (ANN) vytvářejí SVM ve vstupním prostoru hyperrovinu, která slouží jako funkce f odpovědná za mapování vstupů na cílové výstupy. Na rozdíl od ANN, které minimalizují nesoulad mezi výstupem modelu a cílovým výstupem pomocí gradientního sestupu (zpětného šíření), se však SVM snaží vytvořit hyperrovinu, která maximalizuje rozpětí k nejbližšímu datovému bodu libovolné třídy. Tato hyperplocha s maximální marží účinně odděluje datové body (x_i) třídy od štítků (y_i) označené jako 1 od bodů označených jako -1 v rámci souboru dat obsahujícího n vzorků. Zjednodušeně řečeno, hyperplocha maximalizuje vzdálenost k nejbližšímu datovému bodu (x_i) z kterékoli třídy. Obecně je definována jako $\mathbf{w} \cdot \mathbf{x} - b = 0$, kde $\mathbf{w}$ představuje váhový (normálový) vektor hyperplochy a $\frac{b}{\|\mathbf{w}\|}$ určuje posun nebo vychýlení hyperplochy od počátku, SVM používají vstupní atributy $\mathbf{x}$ z trénovacího souboru dat k vytvoření této hyperplochy (viz obrázek 2.2).



Obrázek 2.2: Příklad hyperplochy s maximální hranicí (červená tlustá čára) a hranicemi (černé čáry) pro SVM, která je natrénována na vzorcích dat ze dvou tříd. Plné a prázdné kruhy odpovídají datům se značkami 1, resp. -1. Klasifikace je v tomto příkladu mapována na dvourozměrný vstupní vektor (x_1, x_2) . Dva vzorky dat na okraji - kruhy znázorněné červeným obrysem - jsou podpůrné vektory.[23]

Formálně vzato je tedy SVM funkce $f(x; w, b)$, která předpovídá cílové výstupy (y) a snaží se:

$$\text{minimalizovat } ||\mathbf{w}'||, \text{ podléhá } y_i(\mathbf{w}' \cdot x_i - b) \geq 1, \text{ pro } i = 1, \dots, n \quad (2.1)$$

Parametry klasifikátoru SVM, váhy (w) a zkreslení (b), určují jeho funkčnost. Datové body (vektory x_i), které se nacházejí nejbližší k určené hyperrovině, se označují jako podpůrné vektory. Daná úloha je považována za řešitelnou, pokud trénovací data vykazují lineární separabilitu, což se často označuje jako klasifikační úloha s tvrdou hranicí (viz obr. 2.2). Pokud data nejsou lineárně separovatelná ("soft-margin"), SVM se místo toho pokusí:

$$\text{minimalizovat } \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}' \cdot x_i - b)) \right] + \lambda ||\mathbf{w}'||^2 \quad (2.2)$$

Výraz $\lambda ||\mathbf{w}'||^2$ je výsledkem, pokud jsou splněna tvrdá omezení uvedená v rovnici 2.1, což znamená, že všechny datové body jsou správně klasifikovány na pravé straně okraje. Hodnota vypočtená v rovnici 2.2 je přímo úměrná vzdálenosti od okraje pro nesprávně klasifikovaná data, přičemž λ slouží ke kvalitativnímu určení rozsahu, v jakém by se měla velikost okraje zvětšit, a zároveň zajišťuje, že datové body (x_i) zůstanou na správné straně okraje. Volba malé hodnoty λ se blíží klasifikátoru s tvrdou marží pro lineárně separovatelná data.

V konvenční metodice pro trénování klasifikátorů "soft-margin" je proces učení chápán jako problém kvadratického programování, který vyžaduje prozkoumání prostoru parametrů (w a b) s cílem zjistit nejširší marži, která zahrnuje všechny datové body. Mezi alternativní metodiky patří subgradientní sestup a souřadnicový sestup.

Kromě lineární klasifikace SVM umožňuje i nelineární klasifikaci s využitím různých nelineárních jader. Tato jádra usnadňují mapování vstupního prostoru na prostor vyšších rozměrů, čímž rozšiřují použitelnost SVM. Navzdory této transformaci zůstává základní cíl SVM zachován, i když s nahrazením bodových součinů nelineárními funkcemi jádra. Tato úprava umožňuje algoritmu vymezit hyperplochu s maximální hranicí v transformovaném prostoru příznaků. Mezi nejznámější jádra používaná ve spojení s SVM patří polynomiální funkce, Gaussovy radiální báze funkce a hyperbolické tangenciální funkce.

Zatímco původně byly SVM navrženy pro řešení binární klasifikace, vznikla řada iterací SVM pro klasifikaci a regresi více tříd. SVM mají oproti alternativním SML několik výhod. Zejména vykazují účinnost při řešení problémů spojených s velkými, ale řídkými soubory dat, protože jejich účinnost závisí pouze na podpůrných vektorech pro konstrukci hyperploch. Kromě toho si SVM dovedou poradit s rozsáhlými prostory příznaků, protože složitost jejich učební úlohy není ovlivněna dimenzionalitou prostoru příznaků. Dále SVM představují jednoduchý konvexní optimalizační problém, který zajišťuje konvergenci k singulárnímu globálnímu řešení. A konečně, klasifikační přístup se "soft-margin" umožňuje snadnou kontrolu nad tendencemi k nadměrnému přizpůsobování.

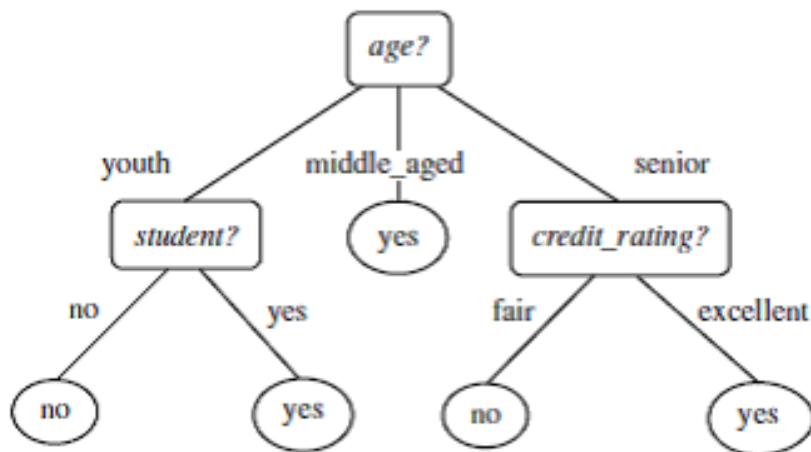
2.1.2 Regrese

Na rozdíl od klasifikace se regrese zaměřuje na předpovídání spojitých cílových proměnných, které představují číselné hodnoty. Jedná se například o předpovídání ceny domu s ohledem na faktory jako je velikost, poloha a vybavení, nebo o odhad prodeje výrobků. Regresní algoritmy jsou navrženy tak, aby se naučily mapování mezi vstupními prvky a spojitými číselnými hodnotami.

Decision Trees(Rozhodovací stromy)

V rámci rozhodovacích stromů(DT)[24] má zkoumaná funkce f strukturu rozhodovacího stromu, který mapuje atributy pozorování dat na odpovídající cílové hodnoty. Vstupy jsou reprezentovány jako uzly, zatímco výstupy jsou ve stromu znázorněny jako listy. Potenciální hodnoty každého vstupního uzlu jsou vymezeny různými větvemi vycházejícími z tohoto uzlu. Analogicky k jiným SML jsou rozhodovací stromy rozděleny do kategorií podle typu výstupních dat, která se snaží naučit. Rozhodovací stromy lze zejména členit na klasifikační nebo regresní podle toho, zda cílový výstup obsahuje konečnou množinu hodnot, nebo spojitou množinu spojitých (intervalových) hodnot.

Příklad rozhodovacího stromu je znázorněn na obrázku 2.3. V tomto stromovém zobrazení odpovídají uzly vstupním atributům a větve se rozšiřují na potomky pro každou myslitelnou hodnotu každého vstupního atributu. Následně listy označují výstupní hodnoty, v tomto případě ohraničují výsledky zda je jedinec studentem nebo zda má příznivé kreditní skóre. Tyto výsledky jsou závislé na hodnotách vstupních atributů, jak je určuje cesta, kterou procházíme od kořene k listu.



Obrázek 2.3: Příklad DT: Vzhledem k věku, statusu studenta a skóre kreditní karty strom předpovídá, zda osoba vlastní počítač. Uzly stromu (zaoblené obdélníky) představují atributy dat neboli vstupy, zatímco listy (ovály) představují cílové hodnoty neboli výstupy. Větve stromu představují možné hodnoty příslušného nadřazeného uzlu stromu.[25]

Cílem DT je formulovat mapování reprezentované jako stromový model, který je schopen předpovídat cílové výstupní hodnoty na základě více vstupních atributů. Základní a široce používaná metoda učení DT z dat je charakterizována hladovým algoritmem. Tento algoritmus předpokládá, že vstupní atributy i cílové výstupy mají konečné diskrétní oblasti a vykazují kategoriální charakteristiky. V případech, kdy vstupy nebo výstupy obsahují spojité hodnoty, může konstrukci stromu předcházet diskretizace. Iterační konstrukce stromu zahrnuje rozdělení dostupné trénovací množiny dat na podmnožiny, která se řídí výběrem atributů. Tento rekursivní proces se odvíjí atribut po atributu a postupně zpřesňuje strukturu stromu.

Různé iterace výše uvedeného postupu vedou k různým algoritmům rozhodovacích stromů. V oblasti učení rozhodovacích stromů se zejména používají dvě významné varianty: Iterative Dichotomiser 3 (ID3) a jeho nástupce C4.5.

2.2 „Učení bez učitele“ neboli Unsupervised ML

TBATM

2.3 „Částečně řízené učení“ neboli Semi-Supervised ML

TBA™

2.4 „Zpětnovazebné učení“ neboli Reinforcement Learning

TBA™

Kapitola 3

Návrh autonomního agenta

TBA[™]

3.1 Prostředí pro agenta

TBA[™]

3.2 Různé algoritmy pro učení agenta

TBA™

3.2.1 Pomocí SARSA (State-Action-Reward-State-Action)

TBA™

3.2.2 Pomocí Deep Q-Learning

TBA™

Kapitola 4

Různé návrhy hodnocení agenta

TBATM

4.1 Podle efektivnosti zabití nepřátel

TBATM

4.2 Podle efektivnosti přežití

TBATM

4.3 Efektivita zabití + přežití

TBATM

Závěr

Literatura

- [1] SAMUEL, Arthur Lee. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*. 1959, roč. 3, č. 3, s. 210-229. DOI 10.1147/rd.33.0210.
- [2] SCHAEFFER, Jonathan; BURCH, Neil; BJÖRNSSON, Yngvi; KISHIMOTO, Akihiro; MÜLLER, Martin et al. Checkers Is Solved. *Science*. 2007, roč. 317, č. 5844, s. 1518-1522. DOI 10.1126/science.1144079.
- [3] Wikipedia contributors. *TD-Gammon*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: <https://en.wikipedia.org/w/index.php?title=TD-Gammon&oldid=1208876151>. [cit. 2024-02-07].
- [4] Wikipedia contributors. *Deep Blue*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: [https://en.wikipedia.org/w/index.php?title=Deep_Blue_\(chess_computer\)&oldid=1208692792](https://en.wikipedia.org/w/index.php?title=Deep_Blue_(chess_computer)&oldid=1208692792). [cit. 2024-02-07].
- [5] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2024-02-07].
- [6] Oppy, Graham and Dowe a David. *The Turing Test*. Online. Stanford Encyclopedia of Philosophy. Winter 2021. Dostupné z: <https://plato.stanford.edu/archives/win2021/entries/turing-test/>. [cit. 2024-02-08].
- [7] GILL, Arthur. *Introduction to the Theory of Finite-State Machines*. 1. McGraw Hill, 207n. 1. ISBN 9780070232433.
- [8] COLLEDANCHISE, Michele a OGREN, Petter. *Behavior Trees in Robotics and AI: An Introduction*. Online. CRC Press, 2018. ISBN 9781138593732. Dostupné z: <https://doi.org/10.1201/9780429489105>. [cit. 2024-02-26].
- [9] ISLA, Damian. *Handling Complexity in the Halo 2 AI*. Online. In: Game Developer. C2024. Dostupné z: <https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-i-halo-2-i-ai>. [cit. 2024-02-10].
- [10] GRAHAM, David „Rez“ a RABIN, Steve. An Introduction to Utility Theory. Online. In: *Game AI Pro 360: Guide to Architecture*. CRC Press, 2019, s. 67-80. ISBN 9780429055058. Dostupné z: <https://doi.org/10.1002/9780429055058.ch4>. [cit. 2024-02-07].

- [//www.taylorfrancis.com/chapters/edit/10.1201/9780429055058-6/introduction-utility-theory-david-rez-graham](https://www.taylorfrancis.com/chapters/edit/10.1201/9780429055058-6/introduction-utility-theory-david-rez-graham). [cit. 2024-02-10].
- [11] DILL, Kevin a MARK, Dave. *Improving AI Decision Modeling Through Utility Theory*. Online. In: GDCVault. 1986. Dostupné z: <https://www.gdcvault.com/play/1012410/Improving-AI-Decision-Modeling-Through>. [cit. 2024-02-10].
 - [12] Wikipedia contributors. *Tree traversal*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=Tree_traversal&oldid=1205867095. [cit. 2024-02-10].
 - [13] YU, Xinjie a GEN, Mitsuo. *Introduction to Evolutionary Algorithms*. Online. Springer London, 2010. ISBN 978-1-84996-129-5. Dostupné z: <https://doi.org/https://doi.org/10.1007/978-1-84996-129-5>. [cit. 2024-02-10].
 - [14] GAMESPOT STAFF. *F.E.A.R. Designer Diary #1 - A Study of Smart AI, Part I*. Online. GameSpot. C2024. Dostupné z: <https://www.gamespot.com/articles/fear-designer-diary-1-a-study-of-smart-ai-part-i/1100-6133519/>. [cit. 2024-02-11].
 - [15] GAMESPOT STAFF. *F.E.A.R. Designer Diary #2 - A Study of Smart AI, Part II*. Online. GameSpot. C2024. Dostupné z: <https://www.gamespot.com/articles/fear-designer-diary-2-a-study-of-smart-ai-part-ii/1100-6134022/>. [cit. 2024-02-11].
 - [16] FIKES, Richard E. a NILSSON, Nils J. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*. Winter 1971, roč. 2, č. 3-4, s. 189-208. ISSN 0004-3702.
 - [17] ORKIN, Jeff. *Three States and a Plan: The A.I. of F.E.A.R.* Online. In: Semantic Scholar. 2015. Dostupné z: <https://api.semanticscholar.org/CorpusID:62493110>. [cit. 2024-02-11].
 - [18] PARKIN, Jeffrey. *Middle-earth: Shadow of War guide: The Nemesis System*. Online. In: Polygon. C2024. Dostupné z: <https://www.polygon.com/middle-earth-shadow-of-war-guide/2017/10/9/16439610/the-nemesis-system-and-you>. [cit. 2024-03-12].
 - [19] Game Maker's Toolkit. *How the Nemesis System Creates Stories*. Online. In: Youtube. C2005-2024. Dostupné z: https://www.youtube.com/watch?v=Lm_AzK27mZY. [cit. 2024-03-12].
 - [20] RUSSELL, Stuart J. a NORVIG, Peter. *Artificial Intelligence : A Modern Approach*. Englewood Cliffs: Prentice Hall, c1995. ISBN 9780131038059.
 - [21] GHORBANI, Fardin; SHABANPOUR, Javad; BEYRAGHI, Sina; SOLEIMANI, Hossein; ORAIZI, Homayoon et al. A deep learning approach for inverse design of the metasurface for dual-polarized waves. Online. *Applied Physics A*. 2021, roč. 127, č. 869, s. 3. ISSN 1432-0630. Dostupné z: <https://doi.org/10.1007/s00339-021-05030-6>. [cit. 2024-04-09].

- [22] STEINWART, Ingo a CHRISTMANN, Andreas. *Support Vector Machines*. Information Science and Statistics. New York, NY: Springer New York, 2008. ISBN 978-0-387-77242-4.
- [23] CHEN, Nongtian; SUN, Youchao; WANG, Zongpeng a PENG, Chong. Improved LS-SVM Method for Flight Data Fitting of Civil Aircraft Flying at High Plateau. Online. *Electronics*. 2022, roč. 11, č. 10, s. 12. ISSN 2079-9292. Dostupné z: <https://doi.org/10.3390/electronics11101558>. [cit. 2024-05-08].
- [24] BREIMAN, Leo; FRIEDMAN, Jerome; OLSHEN, R.A. a STONE, Charles J. *Classification and Regression Trees*. Online. New York: Chapman and Hall/CRC, 2017. ISBN 9781315139470. Dostupné z: <https://doi.org/https://doi.org/10.1201/9781315139470>. [cit. 2024-05-08].
- [25] BOHRA, Aman. *CLASSIFICATION BY DECISION TREE INDUCTION*. Online. In: SPPU BE Computer Laboratory(I,II,III,IV) And Project RELATED STUDY MATERIAL. 2016. Dostupné z: <https://becomputerlaboratory.blogspot.com/2016/10/classification-by-decision-tree.html?m=1>. [cit. 2024-05-08].